

Rapport de Cloud Engineering

Kafka, OpenSearch, NiFi, BigQuery et BigQuery ML

Auteurs : Juan, Victoria, Tasnim, Jorel, Mélina, Amel
Formation : M2 MOSEF

23 mars 2026

Table des matières

Résumé	3
1 Introduction	4
2 Vue d'ensemble de l'architecture	5
2.1 Parcours 1 : flux de données	5
2.2 Parcours 2 : stockage et analytique cloud	5
3 Pipeline Kafka et OpenSearch	6
3.1 Problématique	6
3.2 Rôle du producteur	6
3.3 Rôle du consommateur	7
3.4 Création du schéma OpenSearch	8
3.5 Sélection de l'index et création de l'index pattern	8
3.6 Visualisation dans OpenSearch	9
3.7 Automatisation avec NiFi	9
4 Stockage et analyse dans Google Cloud Platform	10
4.1 Objectif de la partie GCP	10
4.2 Chargement de données dans BigQuery	10
4.3 Vérification du chargement	11
4.4 Création du modèle K-means dans BigQuery ML	11
4.5 Création d'une vue analytique	12
4.6 Consultation des résultats	13
5 Apports techniques et pédagogiques	14
5.1 Maîtrise de la chaîne de données	14
5.2 Complémentarité entre streaming et analytique	14
5.3 Dimension métier	14
6 Limites et pistes d'amélioration	15
6.1 Qualité et gouvernance des données	15
6.2 Industrialisation du pipeline	15
6.3 Analyses enrichies	15
7 Conclusion	16
A Annexe – Structure conseillée pour l'oral ou l'envoi du rapport	17

B Annexe – Version courte de l’architecture

18

Résumé

Ce rapport décrit une chaîne complète de traitement de données dans un contexte cloud. La première partie porte sur la circulation de données en flux, depuis un producteur Kafka jusqu'à leur indexation dans OpenSearch, puis leur visualisation dans un tableau de bord interactif. Une automatisation complémentaire est également introduite à l'aide de NiFi afin d'orchestrer le pipeline sans dépendre exclusivement de scripts manuels. La seconde partie s'intéresse au stockage analytique dans Google Cloud Platform, avec l'insertion de données JSON dans BigQuery, leur vérification, puis l'exploitation de BigQuery ML pour construire un modèle de clustering K-means. Enfin, une vue analytique permet de calculer le chiffre d'affaires par zone géographique et par type de produit.

L'intérêt pédagogique du projet réside dans l'articulation de deux approches complémentaires : le *streaming* pour l'observation et la visualisation quasi temps réel, et l'analytique SQL/cloud pour l'exploitation agrégée, la modélisation et l'aide à la décision. L'ensemble illustre une architecture cohérente de traitement de données, depuis l'ingestion jusqu'à la restitution métier.

Chapitre 1

Introduction

Les environnements cloud modernes permettent de traiter des données selon plusieurs modalités. Certaines applications exigent un traitement en flux, où les événements doivent être collectés, transportés, indexés et rendus visibles presque immédiatement. D'autres nécessitent au contraire une capacité analytique robuste, fondée sur le stockage, le requête SQL, les agrégations et la modélisation. Dans ce projet, ces deux dimensions sont abordées de manière complémentaire.

L'objectif général est double :

- mettre en place une architecture de *data streaming* capable d'acheminer des messages JSON jusqu'à un outil de visualisation ;
- stocker des données dans le cloud, les interroger avec SQL et produire une analyse enrichie à l'aide de BigQuery ML.

Le travail présenté dans les slides s'organise donc autour de deux blocs fonctionnels :

1. **Kafka & OpenSearch** : ingestion d'événements, structuration des données et visualisation ;
2. **Google Cloud Platform / BigQuery** : chargement de données, requêtes SQL, clustering géographique et création d'une vue analytique.

Cette complémentarité est particulièrement intéressante dans un cadre de formation, car elle montre que le cloud ne se limite ni à l'infrastructure ni au stockage, mais constitue un écosystème complet d'ingestion, de transformation, d'analyse et de restitution.

Chapitre 2

Vue d'ensemble de l'architecture

Le projet repose sur une architecture en deux parcours de traitement.

2.1 Parcours 1 : flux de données

Dans le premier parcours, un script Python joue le rôle de **producteur**. Il génère ou transmet des messages au format JSON vers un **topic Kafka**. Un deuxième script Python agit comme **consommateur** : il lit les messages disponibles dans le topic, puis les envoie vers **OpenSearch**. Une fois les données indexées, elles deviennent consultables dans OpenSearch via un schéma explicite, un index pattern et un tableau de bord de visualisation.

Ce parcours répond à une logique temps réel : il s'agit de suivre des événements dès leur émission, de les rendre consultables rapidement et de proposer un support de visualisation interactif.

2.2 Parcours 2 : stockage et analytique cloud

Dans le second parcours, les données sont insérées dans **BigQuery** au format JSON. Après vérification de leur chargement, elles servent de base à des requêtes d'analyse et à un modèle de clustering **K-means** construit avec **BigQuery ML**. Les résultats du clustering permettent ensuite de créer une **vue analytique** regroupant les ventes par zone et par type de produit.

Cette seconde chaîne est davantage orientée vers l'analyse métier : elle vise à structurer les données, calculer des indicateurs synthétiques et dégager des zones géographiques de performance.

Chapitre 3

Pipeline Kafka et OpenSearch

3.1 Problématique

La question de départ posée dans les slides est la suivante : « comment faire des visualisations à partir de données en flux ? » La réponse proposée consiste à construire une architecture simple mais complète, reposant sur Kafka pour le transport des messages et sur OpenSearch pour l'indexation et la visualisation.

3.2 Rôle du producteur

Le producteur est le premier maillon du pipeline. Son rôle est d'émettre des messages au format JSON vers un topic Kafka. Ces messages peuvent représenter des événements, des ventes, des positions géographiques ou toute autre information structurée utile à la démonstration.

L'utilisation du format JSON présente plusieurs avantages :

- il est lisible ;
- il est flexible ;
- il s'intègre facilement avec les outils de streaming et d'indexation.

Un exemple simplifié de producteur Python est donné ci-dessous.

Listing 3.1 – Exemple simplifié de producteur Kafka en Python

```
from kafka import KafkaProducer
import json
import time
from datetime import datetime

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

topic_name = 'streaming-sales'

message = {
```

```
"location": "14.76, -14.76",
"prix": 10.5,
"typeproduit": "fruit",
"agent_timestamp": datetime.utcnow().isoformat()
}

producer.send(topic_name, value=message)
producer.flush()
print("Message envoye vers Kafka")
```

Ce type de script illustre l'idée principale de la slide : un producteur envoie un message JSON dans un topic Kafka, ce qui alimente le pipeline en événements.

3.3 Rôle du consommateur

Le consommateur lit les messages présents dans le topic Kafka et les transmet vers OpenSearch. Il sert donc d'interface entre la couche de transport des données et la couche d'indexation / visualisation.

Ses responsabilités principales sont les suivantes :

1. écouter le topic Kafka ;
2. récupérer les messages entrants ;
3. préparer les données dans un format compatible avec OpenSearch ;
4. indexer chaque message dans un index dédié.

Un schéma de consommateur Python peut être formulé ainsi :

Listing 3.2 – Exemple simplifié de consommateur Kafka vers OpenSearch

```
from kafka import KafkaConsumer
from opensearchpy import OpenSearch
import json

consumer = KafkaConsumer(
    'streaming-sales',
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

client = OpenSearch(
    hosts=[{"host": "localhost", "port": 9200}],
    http_compress=True
)

index_name = 'streaming-sales-index'

for msg in consumer:
    document = msg.value
    client.index(index=index_name, body=document)
    print("Document indexe dans OpenSearch")
```


Ce script correspond à la logique présentée dans les slides : les données stockées dans le topic Kafka sont lues, puis envoyées vers OpenSearch où elles sont stockées dans un index.

3.4 Création du schéma OpenSearch

Une étape essentielle consiste à définir le **mapping** des champs dans OpenSearch. Cette étape est importante, car elle conditionne la manière dont les données seront stockées, recherchées et visualisées.

Dans la présentation, plusieurs types de champs sont mentionnés, notamment :

- `geo_point` pour les coordonnées géographiques ;
- `long` pour des valeurs numériques entières ;
- `text` pour les chaînes de caractères ;
- `date` pour les horodatages.

Un mapping possible est le suivant :

Listing 3.3 – Exemple de mapping OpenSearch

```
PUT streaming-sales-index
{
  "mappings": {
    "properties": {
      "location": { "type": "geo_point" },
      "prix": { "type": "float" },
      "typeproduit": { "type": "text" },
      "agent_timestamp": { "type": "date" }
    }
  }
}
```

Le fait de typer correctement les champs permet :

- d'améliorer les performances de recherche ;
- de faciliter la création de filtres ;
- de rendre possibles des visualisations géographiques ou temporelles ;
- d'éviter certaines ambiguïtés liées au typage automatique.

3.5 Sélection de l'index et création de l'index pattern

Une fois les données indexées, il faut les rendre exploitables dans l'interface OpenSearch. Pour cela, on crée un **index pattern**, c'est-à-dire une configuration qui indique à l'outil quel index utiliser et quel champ temporel servira de référence.

Cette étape est indispensable pour pouvoir :

- naviguer dans les documents ;
- filtrer les données ;
- trier les événements ;
- construire des visualisations fondées sur le temps.

Dans ce projet, le champ temporel naturel est `agent_timestamp`, car il représente le moment d'émission ou d'arrivée de l'événement.

3.6 Visualisation dans OpenSearch

Une fois le schéma et l'index pattern créés, les données deviennent consultables dans l'espace **Discover** d'OpenSearch. Cette interface permet d'explorer les événements, de vérifier l'arrivée des messages et d'observer leur évolution dans le temps.

La logique du tableau de bord est la suivante :

- afficher les événements entrants ;
- filtrer selon certaines colonnes ;
- suivre l'évolution temporelle ;
- produire des visualisations interactives.

OpenSearch joue donc ici un double rôle : moteur d'indexation et couche de restitution visuelle.

3.7 Automatisation avec NiFi

Les slides introduisent également **NiFi** comme outil d'automatisation du pipeline. L'intérêt de NiFi est de pouvoir configurer un flux de traitement par interface graphique, sans avoir à coder chaque étape.

Dans ce contexte, NiFi permet de mettre en place un flux automatisé **Kafka** → **OpenSearch**. Cela présente plusieurs avantages :

- meilleure lisibilité du pipeline ;
- orchestration plus simple des connexions ;
- réduction du besoin de scripts intermédiaires ;
- supervision visuelle du flux.

Le déroulement conceptuel est le suivant :

1. démarrage de NiFi ;
2. création des processeurs de lecture/écriture ;
3. configuration des connexions aux sources et aux destinations ;
4. exécution et supervision du flux.

Ainsi, le projet montre à la fois une implémentation par code et une alternative low-code/no-code pour automatiser le transport des données.

Chapitre 4

Stockage et analyse dans Google Cloud Platform

4.1 Objectif de la partie GCP

La deuxième grande question du projet est la suivante : « comment stocker des données dans le cloud et les analyser avec SQL ? » Pour y répondre, les slides s'appuient sur **BigQuery**, service d'entrepôt de données de Google Cloud Platform, ainsi que sur **BigQuery ML** pour l'aspect modélisation.

L'objectif n'est plus uniquement de visualiser des événements en flux, mais de structurer des données, les interroger et en extraire des indicateurs analytiques.

4.2 Chargement de données dans BigQuery

Les données sont insérées dans une table BigQuery au format JSON. Dans les slides, la table mentionnée est `mosefadata.streaming sell`. Le script d'insertion ajoute des lignes et confirme le nombre de lignes insérées, ce qui constitue une première validation du chargement.

Un exemple cohérent avec la démonstration est présenté ci-dessous.

Listing 4.1 – Exemple de chargement JSON dans BigQuery

```
from google.cloud import bigquery
from google.oauth2 import service_account
from datetime import datetime, UTC

project_id = "juancloudproject"
table_id = "juancloudproject.mosefadata.streaming sell"
key_path = "/content/juancloudproject.json"

credentials = service_account.Credentials.from_service_account_file(key_path)
client = bigquery.Client(credentials=credentials, project=project_id)

rows_to_insert = [
    {
```

```
        "location": "14.76, -14.76",
        "prix": 10.5,
        "typeproduit": "fruit",
        "agent_timestamp": datetime.now(UTC).isoformat()
    },
    {
        "location": "10.76, -14.76",
        "prix": 20.75,
        "typeproduit": "legume",
        "agent_timestamp": datetime.now(UTC).isoformat()
    }
]

errors = client.insert_rows_json(table_id, rows_to_insert)

if not errors:
    print("Insertion reussie")
else:
    print("Erreurs :", errors)
```

Cette étape traduit une logique d'ingestion simple mais fondamentale : les données sont poussées vers une table cloud, qui devient ensuite la base des analyses SQL et du clustering.

4.3 Vérification du chargement

Après l'insertion, la vérification est réalisée dans l'onglet **Preview** de BigQuery. Cette étape, mentionnée explicitement dans les slides, consiste à confirmer que les nouvelles lignes sont bien visibles dans la table. Elle permet de s'assurer que :

- le schéma de la table est compatible avec les données ;
- l'insertion s'est déroulée correctement ;
- les champs sont correctement peuplés ;
- la table peut être exploitée pour les requêtes suivantes.

Cette vérification est souvent sous-estimée, mais elle est cruciale dans un pipeline de données : avant de modéliser ou d'agréger, il faut confirmer que la matière première est correctement chargée.

4.4 Création du modèle K-means dans BigQuery ML

Une des étapes les plus intéressantes du projet est la création d'un modèle de clustering **K-means** dans BigQuery ML. L'idée est de regrouper les ventes selon leur position géographique, à partir des coordonnées extraites de la table `streamingsell`.

Le clustering permet ici d'identifier des **zones géographiques** ou **centroïdes** qui serviront ensuite de support d'analyse métier.

Un exemple de requête SQL compatible avec cette logique est donné ci-dessous.

Listing 4.2 – Creation d'un modele K-means dans BigQuery ML

```
CREATE OR REPLACE MODEL 'mosefadata.mymodel'
OPTIONS(
  MODEL_TYPE = 'KMEANS',
  NUM_CLUSTERS = 4,
  STANDARDIZE_FEATURES = TRUE
) AS
SELECT
  CAST(SPLIT(location, ',')[OFFSET(0)] AS FLOAT64) AS longitude,
  CAST(SPLIT(location, ',')[OFFSET(1)] AS FLOAT64) AS latitude
FROM 'mosefadata.streamingseil';
```

Une fois le modèle créé, les centroïdes peuvent être consultés avec la requête suivante :

Listing 4.3 – Visualisation des centroides du modele

```
SELECT *
FROM ML.CENTROIDS(MODEL 'mosefadata.mymodel');
```

D'un point de vue analytique, cette étape est importante pour plusieurs raisons :

- elle segmente l'espace géographique ;
- elle transforme des coordonnées brutes en zones interprétables ;
- elle prépare la construction d'indicateurs agrégés par cluster.

4.5 Création d'une vue analytique

Après avoir défini les clusters géographiques, les slides indiquent la création d'une vue analytique capable de calculer, pour chaque zone et pour chaque type de produit :

- le chiffre d'affaires total ;
- le nombre de ventes ;
- le prix moyen.

L'idée est d'associer chaque observation à un centroïde, puis d'agréger les résultats pour produire une vision synthétique de la performance commerciale.

Une implémentation SQL cohérente avec cette logique est la suivante :

Listing 4.4 – Vue analytique par zone et type de produit

```
CREATE OR REPLACE VIEW 'mosefadata.v_chiffre_affaire_par_centroid_type' AS
WITH base AS (
  SELECT
    CAST(SPLIT(location, ',')[OFFSET(0)] AS FLOAT64) AS longitude,
    CAST(SPLIT(location, ',')[OFFSET(1)] AS FLOAT64) AS latitude,
    prix,
    typeproduit
  FROM 'mosefadata.streamingseil'
),
pred AS (
  SELECT
    *
```

```
FROM ML.PREDICT(MODEL 'mosefadata.mymodel', TABLE base)
)
SELECT
  CONCAT('Zone ', CAST(centroid_id + 1 AS STRING)) AS zone,
  typeproduit,
  SUM(prix) AS chiffre_affaire_total,
  COUNT(*) AS nombre_ventes,
  AVG(prix) AS prix_moyen
FROM pred
GROUP BY zone, typeproduit;
```

Cette vue constitue un pont entre la modélisation et l'analyse métier. Le clustering seul fournit une segmentation ; la vue analytique lui donne un sens économique.

4.6 Consultation des résultats

La dernière étape consiste à interroger la vue analytique pour afficher les résultats triés par zone puis par chiffre d'affaires décroissant. L'objectif est d'identifier les catégories de produits les plus performantes dans chaque zone.

Une requête typique est la suivante :

Listing 4.5 – Consultation de la vue analytique

```
SELECT *
FROM 'mosefadata.v_chiffre_affaire_par_centroid_type'
ORDER BY zone, chiffre_affaire_total DESC;
```

Cette restitution est particulièrement utile dans une logique d'aide à la décision. Elle permet par exemple de repérer :

- les zones où le volume de ventes est élevé ;
- les produits dominants selon les clusters ;
- les écarts de prix moyens entre zones ;
- les opportunités commerciales ou logistiques.

Chapitre 5

Apports techniques et pédagogiques

Le projet présente plusieurs apports complémentaires.

5.1 Maîtrise de la chaîne de données

L'ensemble du travail montre une compréhension progressive de la chaîne de valeur de la donnée :

1. production d'un événement ;
2. transport par bus de messages ;
3. indexation dans un moteur orienté recherche ;
4. visualisation dans un tableau de bord ;
5. stockage analytique cloud ;
6. modélisation et agrégation.

Cette vision bout-en-bout est particulièrement pertinente dans un enseignement data/-cloud.

5.2 Complémentarité entre streaming et analytique

Le projet montre aussi que **streaming** et **analytique SQL** ne s'opposent pas. Ils répondent à des besoins distincts :

- le streaming permet la réactivité et la visualisation quasi instantanée ;
- l'analytique cloud permet la structuration, l'agrégation et l'interprétation métier.

Dans un système réel, ces deux approches peuvent coexister. Les données circulent d'abord dans un pipeline événementiel, puis alimentent des espaces de stockage plus adaptés aux analyses approfondies.

5.3 Dimension métier

Au-delà de la démonstration technique, l'utilisation d'un indicateur comme le chiffre d'affaires par zone et type de produit apporte une dimension métier claire. Le projet ne se limite pas à montrer des outils : il construit aussi une information utile à l'analyse de performance.

Chapitre 6

Limites et pistes d'amélioration

Même si la démonstration est solide, plusieurs prolongements peuvent être envisagés.

6.1 Qualité et gouvernance des données

Une première amélioration consisterait à renforcer la validation des données dès l'ingestion, par exemple en ajoutant :

- des contrôles de format sur les coordonnées ;
- des règles sur les valeurs de prix ;
- une meilleure gestion des erreurs de transmission ;
- une documentation formelle du schéma.

6.2 Industrialisation du pipeline

Le pipeline pourrait également être davantage industrialisé via :

- une conteneurisation des services ;
- une meilleure séparation entre environnement de test et de production ;
- une supervision plus avancée ;
- une automatisation du déploiement.

6.3 Analyses enrichies

Du côté BigQuery, plusieurs extensions sont possibles :

- intégrer davantage de variables descriptives ;
- comparer plusieurs valeurs de `NUM_CLUSTERS` ;
- ajouter des visualisations externes à partir de la vue créée ;
- construire des tableaux de bord décisionnels connectés à BigQuery.

Chapitre 7

Conclusion

Ce projet de *Cloud Engineering* met en évidence une chaîne de traitement complète, depuis l'émission d'événements jusqu'à leur exploitation analytique. La première partie, fondée sur Kafka et OpenSearch, montre comment des données en flux peuvent être collectées, transportées, structurées et visualisées. La seconde partie, construite autour de BigQuery et BigQuery ML, illustre comment ces données peuvent être stockées dans le cloud, analysées avec SQL et enrichies à l'aide d'un modèle de clustering.

L'intérêt principal du travail réside dans l'articulation entre démonstration technique et lecture métier. Les outils mobilisés ne sont pas présentés isolément : ils s'inscrivent dans un processus cohérent, allant de l'ingestion à l'analyse. La création d'une vue analytique par zone et type de produit constitue à cet égard un bon exemple de transformation d'une donnée brute en indicateur exploitable.

En résumé, le projet illustre efficacement les grands enjeux du cloud data engineering : fluidité des flux, structuration des schémas, stockage scalable, analyse SQL et restitution de résultats utiles à la décision.

Chapitre A

Annexe – Structure conseillée pour l’oral ou l’envoi du rapport

Si ce rapport doit accompagner les slides lors de l’envoi final, une structure de lecture simple peut être retenue :

1. contexte et objectif du projet ;
2. architecture Kafka/OpenSearch ;
3. automatisation avec NiFi ;
4. ingestion dans BigQuery ;
5. clustering K-means ;
6. vue analytique et interprétation ;
7. conclusion et pistes d’amélioration.

Cette organisation évite de dupliquer la présentation visuelle tout en apportant une vraie valeur ajoutée rédactionnelle.

Chapitre B

Annexe – Version courte de l’architecture

Étape	Fonction
Producteur Kafka	Envoie des messages JSON dans un topic.
Consommateur Kafka	Lit les messages du topic et les transmet vers OpenSearch.
OpenSearch Mapping	Définit les types de champs pour indexer correctement les données.
Index Pattern	Rend l’index exploitable dans l’interface de visualisation.
Dashboard / Discover	Permet l’exploration interactive des événements.
NiFi	Automatise le flux Kafka vers OpenSearch sans code manuel.
BigQuery	Stocke les données insérées au format JSON dans le cloud.
BigQuery ML	Crée un modèle K-means à partir des coordonnées.
Vue analytique	Calcule chiffre d’affaires, nombre de ventes et prix moyen par zone et produit.